

# Chapter 4: User Authentication System

Welcome back! In our [previous chapter](#), we learned how the frontend of our interview-ai-*yt* application handles user authentication. We saw how forms let you register and log in, and how the `useAuth` hook and `auth.api.js` service act as the "security desk" and "messengers" to talk to the backend. But what happens *after* the messenger delivers your username and password to the backend?

This is where the **User Authentication System** on the backend comes in!

## What Problem Are We Solving?

Think of our application as a building with different rooms. Some rooms are public (like the homepage), but rooms containing your personal interview reports or tailored resumes are **private**. The "User Authentication System" is the **central security office** or the **bouncer** for this building.

Its job is to:

1. **Verify your identity:** When you try to log in, it checks if you are who you say you are.
2. **Grant access:** If you're verified, it gives you a special "digital pass" (a **token**) that proves you're logged in.
3. **Protect private areas:** For every request to a private room (like fetching your reports), it quickly checks your digital pass to ensure only you can get in.
4. **Manage sessions:** It remembers you're logged in for a certain period, and helps you "log out" to invalidate your pass.

Essentially, it's the backend system that handles all the heavy lifting of user management, security, and access control. It ensures that your data remains private and secure.

## Key Concepts: The Backend's Security Toolbox

To manage all this security, our backend uses a few powerful tools and ideas:

1. **User Data Storage (Database):** First, we need a place to securely store user information like usernames, emails, and *hashed* passwords. We use a database (like MongoDB, which we'll set

up in [Chapter 5: Backend Application Setup](#)) for this.

2. **Password Hashing (bcrypt):** We *never* store your actual password! Instead, we use a special one-way scrambling technique called "hashing" (with a library called **bcrypt**). It converts your password into a long, random-looking string. When you try to log in, we hash your entered password and compare the hashes. If they match, your password is correct, but even if someone saw the hashed password, they couldn't easily unscramble it back to your original password. Think of it like a secret, irreversible code.
3. **JSON Web Tokens (JWT):** This is our "digital pass" or "ID card." When you successfully log in, the backend creates a unique, encrypted string called a JWT. This token contains basic information about you (like your user ID) but *no sensitive data*. It's like a temporary, signed permission slip.
4. **Cookies:** How do you carry this digital pass? We use **cookies**! When the backend gives you a JWT, it tells your browser to store it in a special cookie. Your browser then automatically includes this cookie with every future request you send to our backend. It's like a temporary wristband that you wear, which the bouncer can quickly check.
5. **Middleware:** This is like a **security checkpoint** that sits in front of all private routes. Before any request can reach the part of the code that fetches your reports, the middleware steps in. It checks the cookie for a JWT, verifies if the token is valid, and if so, allows the request to proceed. If not, it blocks access.
6. **Token Blacklisting:** When you log out, we don't just clear your cookie. We also add your JWT to a "blacklist." This tells our middleware: "Even if someone somehow tries to use this specific token, it's no longer valid!" It ensures that once you log out, that specific digital pass cannot be used again.

## How Our Backend Handles Authentication

Let's follow a user's journey, from registering to accessing their data, to see how these concepts work together.

### Use Case: Register, Log In, and Access Private Data

#### 1. Registering:

1. You fill out the "Sign Up" form on the frontend.
2. The frontend sends your username, email, and password to the backend (to `/api/auth/register`).
3. The backend's security office (the `registerUserController`) receives this.
4. It **hashes your password** (turns it into a secret code).
5. It saves your username, email, and the *hashed* password in the database.
6. It then creates a **JWT** (your digital pass) and sends it back to your browser, telling it to store it in a **cookie**.
7. The frontend now sees you as logged in.

## 2. Logging In:

1. You fill out the "Login" form on the frontend.
2. The frontend sends your email and password to the backend (to `/api/auth/login`).
3. The backend's `loginUserController` receives this.
4. It finds your user record in the database by email.
5. It **hashes the password you just entered** and compares it to the *hashed password* stored in the database.
6. If they match, success! It creates a new **JWT** and sends it back to your browser as a **cookie**.
7. The frontend now sees you as logged in.

## 3. Accessing Private Data (e.g., your Interview Reports):

1. You click "My Reports" on the frontend.
2. The frontend sends a request to the backend (e.g., to `/api/interview/`).
3. *Crucially*, your browser automatically includes the **JWT cookie** it received during login.
4. Before the request reaches the "get reports" logic, our **authUser middleware (security checkpoint)** kicks in.
5. It checks the cookie, finds the JWT, and verifies if it's a valid, un-blacklisted token.
6. If the token is valid, the middleware lets the request proceed to the "get reports" part of the backend. It also attaches your user ID to the request, so the backend knows *whose* reports to fetch.
7. The "get reports" logic fetches *your* reports from the database and sends them back.
8. If the token is invalid or missing, the middleware immediately blocks the request and sends an "Unauthorized" error.

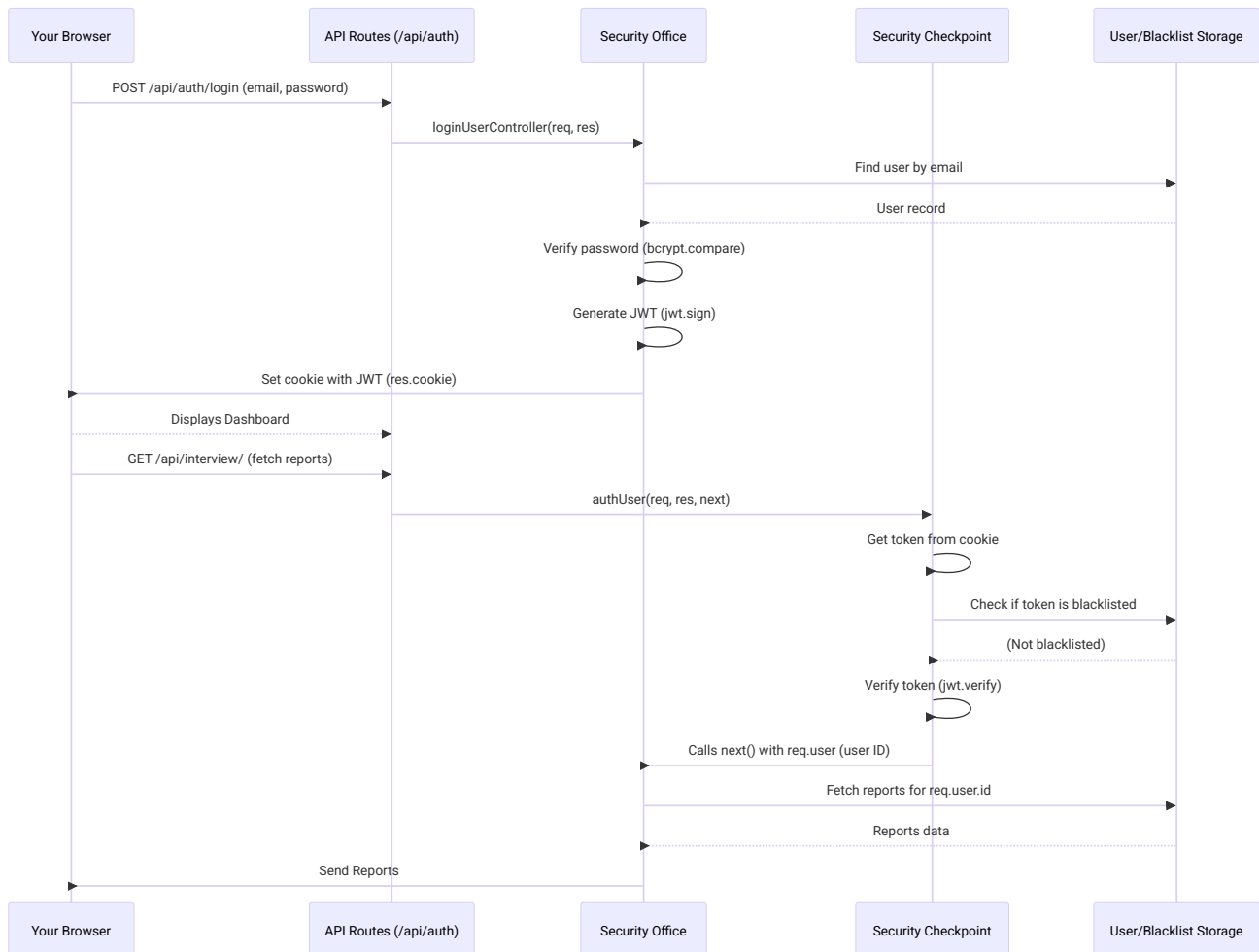
## 4. Logging Out:

1. You click "Logout" on the frontend.
2. The frontend sends a request to the backend (to `/api/auth/logout`).
3. The backend's `logoutUserController` receives this.
4. It takes your current JWT and adds it to the **token blacklist** in the database.
5. It tells your browser to **clear the cookie** containing the JWT.
6. Now, even if someone had your old cookie, the blacklisting ensures it won't work anymore.

# Under the Hood: How the Backend Code Works

Let's look at the actual code files on the backend (Backend/src/) that bring this system to life.

Here's a simplified diagram illustrating the login flow and how the token is then used:



Now, let's dive into the core code snippets that implement these steps.

## 1. User Model (Backend/src/models/user.model.js)

This is the blueprint for how we store user information in our database. Notice that password is stored as a string, but this will be the *hashed* password.

```
// Backend/src/models/user.model.js
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  username: { type: String, unique: true, required: true },
  email: { type: String, unique: true, required: true },
  password: { type: String, required: true } // This will store the HASHED password!
});

const userModel = mongoose.model("users", userSchema);
module.exports = userModel;
```

**Explanation:** This userSchema defines what a user record looks like in our database. username and email must be unique for each user, and all fields are required.

## 2. Registration (Backend/src/controllers/auth.controller.js)

This function handles new user sign-ups.

```
// Backend/src/controllers/auth.controller.js (registerUserController)
const userModel = require("../models/user.model");
const bcrypt = require("bcryptjs"); // For password hashing
const jwt = require("jsonwebtoken"); // For creating tokens

async function registerUserController(req, res) {
  const { username, email, password } = req.body; // 1. Get user input

  // (Error checks for missing fields or existing user omitted for brevity)

  const hash = await bcrypt.hash(password, 10); // 2. HASH the password!

  const user = await userModel.create({ // 3. Save user with hashed password
    username, email, password: hash
  });

  const token = jwt.sign( // 4. Create the digital pass (JWT)
    { id: user._id, username: user.username },
    process.env.JWT_SECRET, // Our secret key for signing
    { expiresIn: "1d" } // Pass is valid for 1 day
  );

  res.cookie("token", token); // 5. Send the pass to browser as a cookie
  res.status(201).json({ message: "User registered successfully", user: { id: user._id, username:
user.username, email: user.email } });
}
```

**Explanation:** After getting the user's details, `bcrypt.hash` scrambles the password. The user is saved to the database. Then, `jwt.sign` creates a JWT, using our secret key (`process.env.JWT_SECRET`) to ensure only our backend can create valid tokens. Finally, `res.cookie` sends this token to the user's browser, telling it to store it.

## 3. Login (Backend/src/controllers/auth.controller.js)

This function handles users logging into existing accounts.

```
// Backend/src/controllers/auth.controller.js (loginUserController)
const userModel = require("../models/user.model");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

async function loginUserController(req, res) {
  const { email, password } = req.body; // 1. Get login input

  const user = await userModel.findOne({ email }); // 2. Find user in database
  if (!user) { return res.status(400).json({ message: "Invalid email or password" }); }

  const isPasswordValid = await bcrypt.compare(password, user.password); // 3. Compare
  entered password with HASHED one
  if (!isPasswordValid) { return res.status(400).json({ message: "Invalid email or password" }); }

  const token = jwt.sign( // 4. Create a new digital pass (JWT)
    { id: user._id, username: user.username },
    process.env.JWT_SECRET,
    { expiresIn: "1d" }
  );

  res.cookie("token", token); // 5. Send the pass to browser as a cookie
  res.status(200).json({ message: "User loggedIn successfully.", user: { id: user._id, username:
user.username, email: user.email } });
}
```

**Explanation:** This function finds the user by email. Then, `bcrypt.compare` takes the plain password entered by the user, hashes it, and compares it with the stored hashed password. If they match, a new JWT is created and sent back as a cookie.

## 4. Authentication Middleware (Backend/src/middlewares/auth.middleware.js)

This is our security checkpoint that runs *before* private routes.

```
// Backend/src/middlewares/auth.middleware.js
const jwt = require("jsonwebtoken");
const tokenBlacklistModel = require("../models/blacklist.model"); // For blacklisted tokens

async function authUser(req, res, next) {
  const token = req.cookies.token; // 1. Get token from the browser's cookie

  if (!token) { return res.status(401).json({ message: "Token not provided." }); } // No pass?
```

Access denied.

```
const isTokenBlacklisted = await tokenBlacklistModel.findOne({ token }); // 2. Check if pass is
revoked
if (isTokenBlacklisted) { return res.status(401).json({ message: "token is invalid" }); }

try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET); // 3. Verify the pass's
signature
  req.user = decoded; // 4. Attach user info (ID) to the request
  next(); // 5. Let the request proceed to the actual route handler
} catch (err) {
  return res.status(401).json({ message: "Invalid token." }); // Invalid pass? Access denied.
}
}
module.exports = { authUser };
```

**Explanation:** authUser gets the token from the req.cookies. It checks our tokenBlacklistModel to see if this token has been explicitly revoked (e.g., after logout). Then, jwt.verify uses our secret key to check if the token is legitimate and hasn't been tampered with. If valid, the user's ID from the token is attached to req.user, and next() is called, allowing the request to continue to its destination.

## 5. Getting Current User Details (Backend/src/controllers/auth.controller.js)

This function (used by the frontend's getMe call) gets details for the currently logged-in user.

```
// Backend/src/controllers/auth.controller.js (getMeController)
const userModel = require("../models/user.model");

async function getMeController(req, res) {
  // Thanks to authUser middleware, req.user already has the user's ID!
  const user = await userModel.findById(req.user.id); // Find user by ID

  res.status(200).json({
    message: "User details fetched successfully",
    user: { id: user._id, username: user.username, email: user.email }
  });
}
```

**Explanation:** Notice how simple this is! Because the authUser middleware ran *before* this controller, req.user.id is already available. The controller just uses this ID to fetch the full user

details from the database.

## 6. Logging Out (Backend/src/controllers/auth.controller.js)

This function handles logging a user out.

```
// Backend/src/controllers/auth.controller.js (logoutUserController)
const tokenBlacklistModel = require("../models/blacklist.model");

async function logoutUserController(req, res) {
  const token = req.cookies.token; // Get the token from the browser

  if (token) {
    await tokenBlacklistModel.create({ token }); // Add token to our blacklist
  }

  res.clearCookie("token"); // Tell the browser to remove the cookie

  res.status(200).json({ message: "User logged out successfully" });
}
```

**Explanation:** When a user logs out, we take their active token and save it in our tokenBlacklistModel (Backend/src/models/blacklist.model.js). This tells our authUser middleware to reject this token if it's ever seen again. We then tell the browser to clearCookie, essentially removing the digital pass.

## 7. Authentication Routes (Backend/src/routes/auth.routes.js)

This file maps specific website addresses (routes) to the controller functions and middleware.

```
// Backend/src/routes/auth.routes.js
const { Router } = require('express');
const authController = require("../controllers/auth.controller");
const authMiddleware = require("../middlewares/auth.middleware");

const authRouter = Router();

authRouter.post("/register", authController.registerUserController); // Public
authRouter.post("/login", authController.loginUserController); // Public
authRouter.get("/logout", authController.logoutUserController); // Public (clears cookie)
authRouter.get("/get-me", authMiddleware.authUser, authController.getMeController); // PRIVATE
```



(requires authUser middleware!)

```
module.exports = authRouter;
```

**Explanation:** This sets up our API endpoints. Notice the /get-me route: it first calls `authMiddleware.authUser` before it calls `authController.getMeController`. This means `authUser` acts as the bouncer, checking the user's digital pass before they can access their details.

## Conclusion

In this chapter, we've dissected the "User Authentication System" on our backend. We uncovered how it securely manages user accounts using password hashing, issues digital passes (JWTs) via cookies, and employs a security checkpoint (middleware) to protect private routes. We also learned how logging out revokes these passes through a token blacklist.

This robust system is the backbone of our application's security, ensuring that your valuable interview data remains accessible only to you. But how do we get this backend application up and running, connect it to a database, and make sure all these services can talk to each other? That's what we'll explore in our next chapter!